Load libraries and set user directory paths

In [ ]:
```r
## Function to execute shell commands from within R
## Useful when running R in Google Colab or other environments where system call
shell_call <- function(command, ...) {
  result <- system(command, intern = TRUE, ...)  # Execute the shell command ana
  cat(paste0(result, collapse = "\n"))  # Print the result in a readable format
}

## Function to load required R packages
## If a package is not installed, it notifies the user
loadPackages = function(pkgs){
  myrequire = function(...){
    suppressWarnings(suppressMessages(suppressPackageStartupMessages(require(...
  }
  ok = sapply(pkgs, require, character.only=TRUE, quietly=TRUE)  # Check if pack
  if (!all(ok)){
    message("There are missing packages: ", paste(pkgs[!ok], collapse=", "))  #
  }
}

## Setup R2U (Ubuntu optimized R package manager) for faster package installatio
download.file("https://github.com/eddelbuettel/r2u/raw/master/inst/scripts/add_c
              "add_cranapt_jammy.sh")  # Download installation script
Sys.chmod("add_cranapt_jammy.sh", "0755")  # Grant execution permissions to the
shell_call("./add_cranapt_jammy.sh")  # Run the script
bspm::enable()  # Enable BSPM (Bridge to System Package Manager) for installing
options(bspm.version.check=FALSE)  # Disable BSPM version check
shell_call("rm add_cranapt_jammy.sh")  # Remove the installation script after ex

## Install necessary R packages from CRAN
cranPkgs2Install = c("dplyr", "ggpubr", "Seurat", "cowplot",
                     "Rtsne", "hdf5r", "patchwork")
install.packages(cranPkgs2Install, ask=FALSE, update=TRUE, quietly=TRUE)
```

In [ ]:
```r
## To simplify package loading, we created the loadPackages() function
## If you don't have this function, you should load packages using library(name_
pkgs = c("Seurat", "dplyr","patchwork","ggplot2")
loadPackages(pkgs)  # Load the specified packages

# Important
# Update scw01 dorectory to match your personal user directory
# To match your personal user directory. This is where you read and write data
mydir <- "/content"
```

Multi-modal Data Analysis

In this notebook, we will embark on the exciting journey of analyzing multi-modal data, specifically focusing on the RNA and protein expression levels in individual cells. We will be guided by a comprehensive tutorial from Seurat, which you can find in detail here.

The dataset we will be working with consists of 8,617 cord blood mononuclear cells (CBMCs), the original article is Simultaneous epitope and transcriptome measurement in

single cells.

These cells have been meticulously analyzed to provide both transcriptomic measurements and abundance estimates for 11 surface proteins. The protein levels have been quantified using DNA-barcoded antibodies, allowing for precise and reliable data.

To begin our analysis, we will load two count matrices: one containing the RNA measurements and the other containing the antibody-derived tags (ADT). This dual dataset will enable us to explore and understand the intricate relationships between RNA expression and protein levels in these individual cells.

Here's a brief overview of our dataset as described by Seurat: > "In this analysis, we examine a dataset of 8,617 cord blood mononuclear cells (CBMCs), where transcriptomic measurements are paired with abundance estimates for 11 surface proteins, quantified using DNA-barcoded antibodies. Initially, we load two count matrices: one for the RNA measurements and one for the antibody-derived tags (ADT)."

By following this tutorial, we aim to gain insights into the biological mechanisms at play and the interactions between RNA and protein expressions at a single-cell level.

Get ready for a deep dive into multi-modal data analysis!

In [3]:
```
# Download raw single-cell RNA-seq and ADT (antibody-derived tag) data from the
shell_call("wget ftp://ftp.ncbi.nlm.nih.gov/geo/series/GSE100nnn/GSE100866/suppl
shell_call("wget ftp://ftp.ncbi.nlm.nih.gov/geo/series/GSE100nnn/GSE100866/suppl
```

In [ ]:
```r
# Load in the RNA UMI matrix
# Note: The dataset also contains ~5% of mouse cells, which we can use as negati
# For this reason, the gene expression matrix has HUMAN_ or MOUSE_ appended to t

cbmc.rna <- as.sparse(read.csv(file = paste0(mydir,"/GSE100866_CBMC_8K_13AB_10X-

# read.csv: reads a CSV file into a data frame, file argument specifies the path
# and (paste0) concatenates the directory stored in mydir with the filename.
# sep = ",": Indicates that the delimiter used in the CSV file is a comma.
# header = TRUE: Specifies that the first row of the CSV file contains column na
# row.names = 1: Specifies that the first column of the CSV file should be used
# as.sparse(): This function converts the data frame into a sparse matrix.
# Sparse matrices are used to efficiently store data with a lot of zero values,

# To make life a bit easier going forward, we're going to discard all but the to
# and remove the 'HUMAN_' from the CITE-seq prefix

cbmc.rna <- CollapseSpeciesExpressionMatrix(cbmc.rna)

# This command processes the cbmc.rna matrix to handle or collapse gene expressi
# ensuring that the matrix is properly formatted for further analysis.

# Load the ADT UMI matrix (for protein-level measurements)
cbmc.adt <- as.sparse(read.csv(file = paste0(mydir,"/GSE100866_CBMC_8K_13AB_10X-

# Note: Ensure that both RNA and ADT matrices have identical column names (i.e.,
# This command used to compare the column names of two data frames

all.equal(colnames(cbmc.rna), colnames(cbmc.adt))  # Should return TRUE
```

In [ ]:
```r
# Create a Seurat object for the scRNA-seq data
cbmc <- CreateSeuratObject(counts = cbmc.rna)

# Verify the available assays (default is RNA).
# The cbmc object contains an assay storing RNA measurement
Assays(cbmc)

# Create a new assay to store ADT information
adt_assay <- CreateAssayObject(counts = cbmc.adt)

# Add the ADT assay to the Seurat object
cbmc[["ADT"]] <- adt_assay

# Verify that the Seurat object now contains both RNA and ADT assays
Assays(cbmc)

# Extract a list of features (antibodies) measured in the ADT assay
rownames(cbmc[["ADT"]])

# Note: We can easily switch back and forth between the two assays to specify th

# List the current default assay (should be RNA)
DefaultAssay(cbmc)

# Switch the default assay to ADT
DefaultAssay(cbmc) <- "ADT"
DefaultAssay(cbmc)  # Should now return "ADT"
```

Clustering

In [ ]:
```
# Note that all operations below are performed on the RNA assay Set and verify t
# Default assay is RNA
DefaultAssay(cbmc) <- "RNA" # This command sets the default assay for the cbmc o
DefaultAssay(cbmc) # Show the Assay

# Perform visualization and clustering steps
cbmc <- NormalizeData(cbmc) # This command normalizes the gene expression data i
cbmc <- FindVariableFeatures(cbmc) # This command identifies most variable featu
cbmc <- ScaleData(cbmc) # This command scales the data, adjusting the mean of ge
cbmc <- RunPCA(cbmc, verbose = FALSE) # This command performs a Principal Compon
cbmc <- FindNeighbors(cbmc, dims = 1:30) # This command finds the nearest neighb
cbmc <- FindClusters(cbmc, resolution = 0.8) # This command performs clustering
cbmc <- RunUMAP(cbmc, dims = 1:30) # This command performs UMAP (Uniform Manifol
DimPlot(cbmc, label = TRUE) # This command generates a scatter plot of the cells
```

Visualize multiple modalities side-by-side

Now that we have successfully clustered our scRNA-seq profiles, we can move on to visualizing the expression of either protein or RNA molecules in our dataset. Seurat offers several methods to switch between different modalities and specify which modality you are interested in analyzing or visualizing.

This is especially important because, in some cases, the same feature can be present in multiple modalities. For instance, in our dataset, we have independent measurements of the B cell marker CD19, both at the protein level and RNA level. Being able to switch between these modalities allows us to comprehensively explore and understand the biological significance of these features.

By leveraging Seurat's capabilities, we can gain deeper insights into the relationships and interactions between RNA and protein expressions within individual cells, ultimately enhancing our understanding of the underlying biological processes.

If you have any questions or need further assistance with the analysis, feel free to ask!

In [ ]:
```
# Normalize ADT data
DefaultAssay(cbmc) <- "ADT" # This command sets the default assay for the cbmc o
cbmc <- NormalizeData(cbmc, normalization.method = "CLR", margin = 2)
# This command normalizes the ADT data in the cbmc object using the CLR (Centere
# The margin = 2 parameter indicates that normalization will be applied to the c

DefaultAssay(cbmc) <- "RNA" # This command resets the default assay for the cbmc

# Note that the following command is an alternative but returns the same result
cbmc <- NormalizeData(cbmc, normalization.method = "CLR", margin = 2, assay = "A

# Now, we will visualize CD14 levels for RNA and protein.
# By setting the default assay, we can visualize one or the other
DefaultAssay(cbmc) <- "ADT"
p1 <- FeaturePlot(cbmc, "CD19", cols = c("lightgrey", "darkgreen")) + # This com
+ ggtitle("CD19 protein") # Add a title to the plot p1
```

```
DefaultAssay(cbmc) <- "RNA" # This command resets the default assay to RNA.
p2 <- FeaturePlot(cbmc, "CD19") + ggtitle("CD19 RNA") # This command creates a F

# Place plots side-by-side
p1 | p2 # Show p1 and p2 side by side


# Alternately, we can use specific assay keys to specify a specific modality Ide
Key(cbmc[["RNA"]])
Key(cbmc[["ADT"]])


# These commands identify the keys for the RNA and ADT assays, respectively. Key
# Now, we can include the key in the feature name, which overrides the default a
p1 <- FeaturePlot(cbmc, "adt_CD19", cols = c("lightgrey", "darkgreen")) + ggtitl
p2 <- FeaturePlot(cbmc, "rna_CD19") + ggtitle("CD19 RNA")
p1 | p2
```

Identify cell surface markers for scRNA-seq clusters

In [ ]:
```
# As we know, CD19 is a B cell marker, we can identify cluster 6 as expressing C
VlnPlot(cbmc, "adt_CD19") # Create a violin plot

# We can also identify alternative protein and RNA markers for this cluster thro
adt_markers <- FindMarkers(cbmc, ident.1 = 6, assay = "ADT") # This function ide
rna_markers <- FindMarkers(cbmc, ident.1 = 6, assay = "RNA") # This function ide
# ident.1 = 6: Specifies that cluster 6 is the group of interest for which marke
# assay: Indicates that the ADT/RNA assay data should be used for the analysis.
# adt/rna_markers: Stores the results of the marker identification for the ADT/R

# Display the top markers
head(adt_markers)
head(rna_markers)
```

Additional visualizations of multimodal data

In [ ]:
```
# Draw ADT scatter plots (like biaxial plots for FACS).
# Note that you can even 'gate' cells if desired by using HoverLocator and Featu
FeatureScatter(cbmc, feature1 = "adt_CD19", feature2 = "adt_CD3")

# View relationship between protein and RNA
FeatureScatter(cbmc, feature1 = "adt_CD3", feature2 = "rna_CD3E")

FeatureScatter(cbmc, feature1 = "adt_CD4", feature2 = "adt_CD8")

# Let's look at the raw (non-normalized) ADT counts. You can see the values are
# particularly in comparison to RNA values. This is due to the significantly hig
# copy number in cells, which significantly reduces 'drop-out' in ADT data

FeatureScatter(cbmc, feature1 = "adt_CD4", feature2 = "adt_CD8", slot = "counts"
# slot = "counts": This parameter specifies that the raw (non-normalized) counts
```

Loading data from 10X multi-modal experiments

NOTE: DO NOT RUN THIS SECTION

We are including it for you reference if you will need to process multi-modal 10x data from the raw data.

In [ ]:

```
# Load the 10X Genomics PBMC dataset from the specified directory
# This data includes gene expression and antibody capture information
pbmc10k.data <- Read10X(data.dir = "../data/pbmc10k/filtered_feature_bc_matrix/"

# Clean up the row names of the Antibody Capture data to remove unwanted suffixe
# specifically the '_TotalSeqB' and any 'control' prefixes
rownames(x = pbmc10k.data[["Antibody Capture"]]) <- gsub(pattern = "_[control_]*
    x = rownames(x = pbmc10k.data[["Antibody Capture"]]))
# rownames(): This function retrieves or sets the row names of a matrix or data
# gsub(): This function performs replacement of patterns in strings. Here, it is
# Start with an underscore followed by "control" (optionally repeated) and end w
# replecement = "": This indicates that the matched pattern should be replaced w
# x = rownames(x = pbmc10k.data[["Antibody Capture"]]): This specifies the input


# Create a Seurat object for the gene expression data,
# filtering to keep genes detected in at least 3 cells and cells with at least 2
pbmc10k <- CreateSeuratObject(counts = pbmc10k.data[["Gene Expression"]], min.ce

# Normalize the gene expression data using log normalization
pbmc10k <- NormalizeData(pbmc10k)

# Add the ADT data (Antibody Derived Tags) to the Seurat object as a separate as
# This ensures we can analyze protein expression alongside RNA data
pbmc10k[["ADT"]] <- CreateAssayObject(pbmc10k.data[["Antibody Capture"]][, colna

# Normalize the ADT data using CLR (Centered Log Ratio) normalization method
# This is important for correcting technical variation in protein expression dat
pbmc10k <- NormalizeData(pbmc10k)

# Add the ADT data (Antibody Derived Tags) to the Seurat object as a separate as
# This ensures we can analyze protein expression alongside RNA data
pbmc10k[["ADT"]] <- CreateAssayObject(pbmc10k.data[["Antibody Capture"]][, colna

# Normalize the ADT data using CLR (Centered Log Ratio) normalization method
# This is important for correcting technical variation in protein expression dat
pbmc10k <- NormalizeData(pbmc10k, assay = "ADT", normalization.method = "CLR")

# Create scatter plots to visualize the relationship between different features:
# Plot 1: CD19 vs. CD3 protein expression
plot1 <- FeatureScatter(pbmc10k, feature1 = "adt_CD19", feature2 = "adt_CD3", pt

# Plot 2: CD4 vs. CD8a protein expression
plot2 <- FeatureScatter(pbmc10k, feature1 = "adt_CD4", feature2 = "adt_CD8a", pt

# Plot 3: CD3 protein expression vs. CD3E RNA expression
plot3 <- FeatureScatter(pbmc10k, feature1 = "adt_CD3", feature2 = "CD3E", pt.siz

# Combine the three scatter plots into one figure and remove the legend for clar
(plot1 + plot2 + plot3) & NoLegend()
```